

Geração de Sinais Analógicos na ESP32

DAC · PWM · Filtros · Técnicas de Alta Resolução

PlatformIO + ESP32 | Sistemas Embarcados

Agenda

1. **Fundamentos** — Digital vs Analógico, Resolução e Quantização
2. **DAC** — Conversor Digital-Analógico na ESP32
3. **PWM** — Modulação por Largura de Pulso
4. **PWM + Filtro RC** — Reconstrução de Sinal Analógico
5. **PWM Dual (2 canais)** — Técnica de Alta Resolução
6. **Comparação DAC vs PWM** — Análise Comparativa
7. **Código Prático** — PlatformIO + ESP32
8. **Aplicações Reais** — Áudio, Motores, Formas de Onda

01

Fundamentos

Digital vs Analógico · Resolução · Quantização

Sinal Digital vs Analógico

Sinal Digital

- Valores discretos: apenas 0 (LOW) ou 1 (HIGH)
- Na ESP32: 0V ou 3.3V
- Transições instantâneas entre estados

Sinal Analógico

- Valores contínuos: qualquer tensão entre 0V e 3.3V
- Variação suave e gradual ao longo do tempo
- Essencial para áudio, controle de motores, sensores

⚠ **Desafio:** Microcontroladores são digitais — como gerar saída analógica?

Resolução e Quantização

Resolução (n bits)

- Número de níveis discretos = 2^n
- 8 bits = 256 níveis (DAC da ESP32)
- Passo de tensão = $V_{\text{ref}} / 2^n = 3.3\text{V} / 256 \approx \mathbf{12.9\text{ mV}}$

Quantização

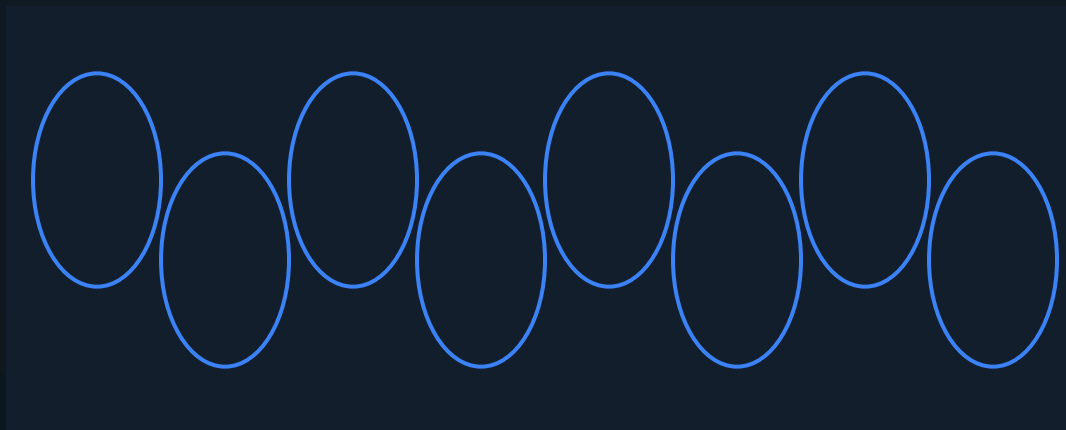
- Processo de mapear valor contínuo para discreto
- Erro de quantização = $\pm 1/2$ LSB ($\pm 6.45\text{ mV}$ no DAC 8-bit)
- Maior resolução $\rightarrow$ menor erro $\rightarrow$ melhor fidelidade

Ruído de Quantização

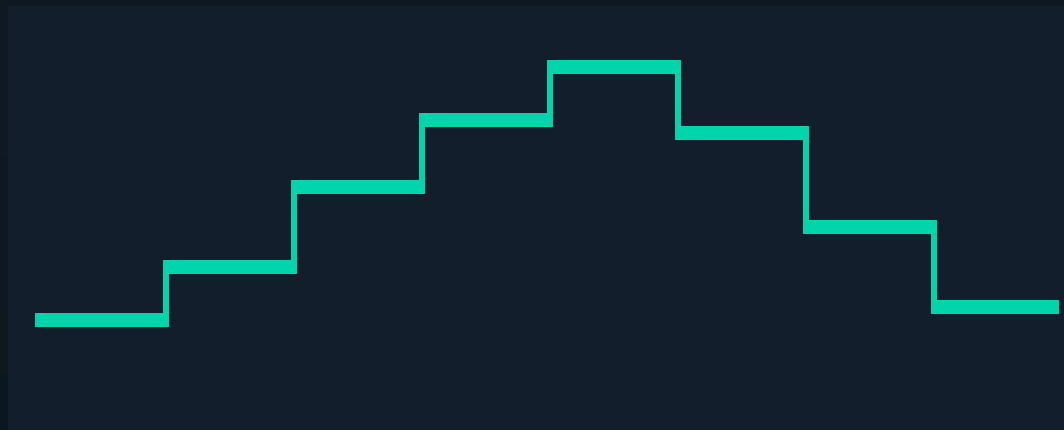
- $\text{SNR} = 6.02n + 1.76\text{ dB} \rightarrow 8\text{ bits} \approx \mathbf{49.9\text{ dB}}$

Visualização: Quantização de Sinal

Sinal Analógico (contínuo)



Sinal Quantizado (discreto)



4 bits

16 níveis

8 bits

256 níveis

10 bits

1024 níveis

16 bits

65536 níveis

↑ Maior resolução = menor erro de quantização = sinal mais fiel

02

DAC — Digital to Analog Converter

Conversor Digital-Analógico na ESP32

O que é um DAC?

Definição

- Converte valor numérico digital em tensão analógica proporcional
- Entrada: valor digital (0–255) → Saída: tensão (0–3.3V)

DAC na ESP32

- 2 canais: **GPIO 25 (DAC1)** e **GPIO 26 (DAC2)**
- Resolução: **8 bits (apenas 256 níveis)**
- Tipo: resistor string (R-2R ladder interno)
- Saída: tensão real contínua (sem filtragem)

$$V_{\text{out}} = (\text{digital_value} / 255) \times 3.3V$$

DAC: Vantagens e Desvantagens

✓ Vantagens

- Saída analógica real (contínua)
- Não requer hardware externo
- Sem ripple (sinal limpo)
- Resposta imediata
- Simples de programar
- Ideal para sinais de baixa precisão

✗ Desvantagens

- Apenas **8 bits** de resolução
- Ruído térmico e de quantização
- Apenas 2 canais disponíveis
- Não-linearidade do DAC interno
- Baixa capacidade de corrente
- Impreciso para aplicações críticas

03

PWM — Pulse Width Modulation

Modulação por Largura de Pulso

Conceito de PWM

Duty Cycle (Ciclo de Trabalho)

- Proporção do tempo em HIGH vs período total
- $\text{Duty} = T_{\text{on}} / T_{\text{total}} \times 100\%$
- $V_{\text{média}} = \text{Duty} \times V_{\text{max}} = \text{Duty} \times 3.3\text{V}$

Frequência vs Resolução

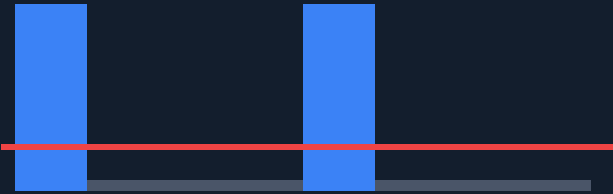
- $f_{\text{PWM}} = f_{\text{clock}} / 2^n$ (tradeoff fundamental)
- Maior resolução → menor frequência máxima
- ESP32 LEDC: até **16 canais, 1-20 bits**

Como simula sinal analógico?

- A tensão média da onda quadrada é proporcional ao duty
- Com filtro passa-baixa: remove componente AC → DC puro

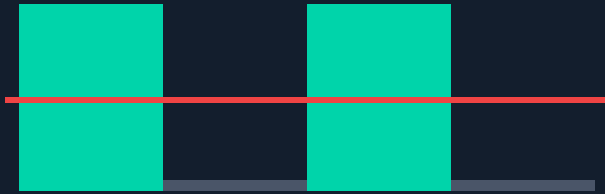
Duty Cycle — Visualização

25% Duty



V média = 0.83V
 $25\% \times 3.3V$

50% Duty



V média = 1.65V
 $50\% \times 3.3V$

75% Duty



V média = 2.48V
 $75\% \times 3.3V$

Fórmula Fundamental

$$V_{\text{média}} = (\text{Duty Cycle} / 100) \times V_{\text{max}} = (T_{\text{on}} / T_{\text{período}}) \times 3.3V$$

⚡ Insight Chave

PWM é um sinal 100% digital (0V ou 3.3V). A "tensão analógica" só existe como média temporal. Para obter um sinal DC real, é necessário filtrar as componentes AC com um filtro passa-baixa.

04

PWM + Filtro RC

Transformando PWM em Sinal Analógico Real

Filtro Passa-Baixa (RC)

Princípio de Funcionamento

- Resistor (R) + Capacitor (C) formam filtro passa-baixa
- Atenua frequências acima de f_c (componente PWM)
- Preserva componente DC (tensão média desejada)

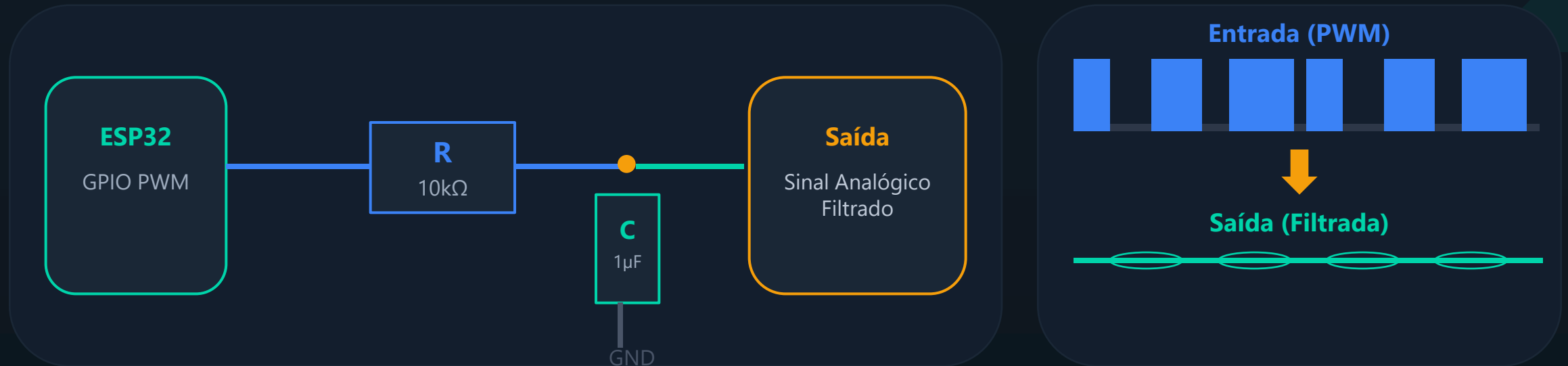
Parâmetros Chave

- Frequência de corte: $f_c = 1 / (2\pi RC)$
- Constante de tempo: $\tau = R \times C$
- Tempo de estabilização $\approx 5\tau$ (99.3% do valor final)

Tradeoff: Suavidade vs Velocidade

- RC grande $\rightarrow$ sinal suave, mas resposta lenta
- RC pequeno $\rightarrow$ resposta rápida, mas ripple residual

Circuito: Filtro RC Passa-Baixa



Regras de Projeto

1. f_c deve ser $\ll f_{PWM}$ (pelo menos 10 $\times$)
2. R típico: 1k Ω a 100k Ω
3. C típico: 100nF a 10 μ F
4. **Exemplo: $R=10k\Omega$, $C=1\mu F \rightarrow f_c \approx 16Hz$**

$f_{PWM} = 5kHz$, $f_c = 16Hz \rightarrow$ atenua $\sim 48dB$

$\tau = 10ms \rightarrow$ estabiliza em $\sim 50ms$

Ripple residual < 1% com bom projeto

PWM com 1 Canal — Geração de Tensão Média

Como Funciona

- 1 canal PWM + filtro RC = tensão DC variável
- Resolução limitada pela resolução do timer PWM
- 8 bits @ 312.5 kHz ou 10 bits @ 78.125 kHz

Limitações

- **Ripple:** oscilação residual na saída (ondulação)
- **Resolução efetiva:** reduzida pelo ruído do filtro
- **Velocidade:** tempo de resposta depende de $\tau = RC$
- **Tradeoff:** $\uparrow$ freq PWM = $\downarrow$ resolução (e vice-versa)

05

PWM com 2 Canais (Alta Resolução)

Técnica de Combinação MSB + LSB para Resolução Estendida

Técnica: PWM Dual (MSB + LSB)

Princípio

- Combinar 2 canais PWM com pesos diferentes
- PWM1 (**MSB**): bits mais significativos → R pequeno
- PWM2 (**LSB**): bits menos significativos → R grande (256×)

Matemática

- $V_{out} = V_{MSB} + V_{LSB} / 256$
- 8 bits MSB + 8 bits LSB = **16 bits efetivos (65.536 níveis!)**
- Passo de tensão: $3.3V / 65536 \approx 50 \mu V!$

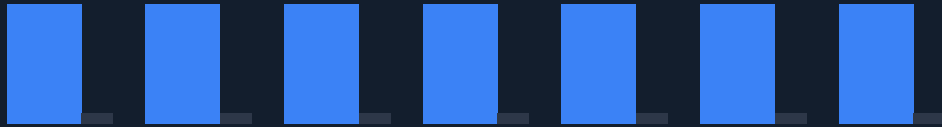
Circuito

- PWM1 → R1 (1kΩ) → nó de soma
- PWM2 → R2 (256kΩ) → nó de soma → C → V_{out}

Diagrama: Combinação de 2 PWMs (MSB + LSB)

PWM1 — MSB (Grossoiro)

8 bits → 256 níveis | R1 = 1kΩ



Contribuição: ~2.31V (70% de 3.3V)

PWM2 — LSB (Fino)

8 bits → 256 níveis | R2 = 256kΩ



Contribuição: ~5.2mV (0.4% × 3.3V / 256)

Rede Resistiva de Soma + Filtro RC

$$V_{\text{out}} = (\text{PWM1} \times R2 + \text{PWM2} \times R1) / (R1 + R2) \approx \text{PWM1}_{\text{filtrado}} + \text{PWM2}_{\text{filtrado}}/256$$

Saída Analógica de Alta Resolução

$$V_{\text{out}} = 2.31\text{V} + 5.2\text{mV} = 2.3152\text{V}$$

Resolução efetiva: 16 bits (65.536 níveis)

Passo mínimo: ~50μV | SNR teórico: ~98 dB

DAC nativo: 8 bits = 256 níveis

PWM 1 canal: ~10 bits = 1024 níveis

PWM 2 canais: 16 bits = 65536 níveis

Ganho de resolução: 256× maior que o DAC nativo!

PWM Dual — Implementação Prática

Estratégia de Resolução

- Valor desejado (16 bits): **MSB = valor >> 8** (bits 15-8)
- Ajuste fino: **LSB = valor & 0xFF** (bits 7-0)
- Razão dos resistores: $R2/R1 = 2^n$ (ex: 256×)

Componentes Necessários

- $R1 = 1k\Omega$ (canal MSB) + $R2 = 256k\Omega$ (canal LSB)
- Capacitor de filtragem: $C = 1\mu F$ (cerâmico)
- 2 GPIOs da ESP32 (LEDC canais 0 e 1)

Precisão dos Resistores

- Usar resistores de **1% de tolerância** (mínimo)
- Erro no ratio $R2/R1$ limita resolução efetiva
- Calibração por software pode compensar imprecisões

06

Comparação: DAC vs PWM

Análise Comparativa dos Métodos de Geração Analógica

DAC vs PWM — Tabela Comparativa

Método	Qualidade	Resolução	Complexidade	Hardware Extra	Uso Ideal
DAC Nativo	Média	8 bits (256)	Baixa	Nenhum	Sinais simples
PWM (1 canal)	Baixa/Média	8-10 bits	Baixa	Filtro RC	Econômico
PWM (2 canais)	Alta	16 bits (65k)	Média	Filtro RC	Alta precisão
DAC Externo (I ² C)	Muito Alta	12-16 bits	Alta	Chip + I ² C	Profissional

★ PWM com 2 canais oferece o melhor custo-benefício para alta resolução na ESP32

Velocidade

DAC > PWM1 > PWM2

Resolução

PWM2 > DAC ext >
PWM1 > DAC

Custo

DAC = PWM1 < PWM2 <
DAC ext

Simplicidade

DAC > PWM1 > PWM2 >
DAC ext

07

Arquitetura da ESP32

Periféricos DAC e LEDC (PWM)

Periféricos de Saída Analógica

DAC (Digital-to-Analog Converter)

- 2 canais independentes (GPIO 25, GPIO 26)
- 8 bits, R-2R ladder, barramento APB (80 MHz)
- Suporta DMA para geração contínua de formas de onda

LEDC (LED Control / PWM)

- 16 canais independentes (8 high-speed + 8 low-speed)
- Resolução configurável: **1 a 20 bits**
- Fonte de clock: APB (80 MHz) ou REF_TICK (1 MHz)
- Frequência máxima: $80 \text{ MHz} / 2^n$ (ex: 8 bits $\rightarrow$ 312.5 kHz)
- Pode ser mapeado para qualquer GPIO via matrix

Geração Interna do Sinal

- Timer conta de 0 até $2^n - 1$ (período)
- Comparador: saída HIGH quando $\text{timer} < \text{duty_value}$

08

Código Prático

PlatformIO + ESP32 | DAC, PWM 1ch, PWM 2ch

Código: DAC — Escrita de Valor Analógico

```
#include <Arduino.h>

#define DAC_PIN 25 // GPIO 25 = DAC1
#define DAC_PIN2 26 // GPIO 26 = DAC2

void setup() {
  Serial.begin(115200);
}

void loop() {
  // Rampa de 0 a 3.3V
  for (int i = 0; i < 256; i++) {
    dacWrite(DAC_PIN, i);
    delay(10);
  }
  // Valor fixo: 1.65V
  dacWrite(DAC_PIN, 128);
  delay(2000);
}
```

Pontos-Chave

dacWrite(pin, valor)

Função nativa do Arduino

valor: 0 a 255

0 = 0V, 128 = 1.65V

255 = 3.3V

Sem configuração!

Não precisa de timer,
canal ou frequência.
Apenas 1 linha de código.

⚠ Limitação:

Apenas GPIO 25 e 26

8 bits = 12.9mV/step

PlatformIO | framework = arduino | board = esp32dev

Código: PWM 1 Canal (LEDC)

```
#include <Arduino.h>

#define PWM_PIN    18
#define PWM_FREQ   5000 // 5kHz
#define PWM_CH     0
#define PWM_RES    8     // 8 bits

void setup() {
    // Configura canal LEDC
    ledcSetup(PWM_CH, PWM_FREQ, PWM_RES);
    // Associa canal ao GPIO
    ledcAttachPin(PWM_PIN, PWM_CH);
}

void loop() {
    // Rampa de duty cycle
    for (int d = 0; d < 256; d++) {
        ledcWrite(PWM_CH, d);
        delay(10);
    }
}
```

API LEDC (3 passos)

1. **ledcSetup(ch, freq, res)**

Configura timer do canal

2. **ledcAttachPin(gpio, ch)**

Conecta GPIO ao canal

3. **ledcWrite(ch, duty)**

Define duty cycle (0-255)

Frequência vs Resolução:

8 bits: até 312.5 kHz

10 bits: até 78.1 kHz

13 bits: até 9.8 kHz

+ Filtro RC na saída ($R=10k\Omega$, $C=1\mu F$) para obter tensão DC

Código: PWM 2 Canais (Alta Resolução)

```
#include <Arduino.h>

#define MSB_PIN 18 // PWM grosseiro
#define LSB_PIN 19 // PWM fino
#define FREQ 5000
#define RES 8 // 8 bits cada

void setup() {
  ledcSetup(0, FREQ, RES); // MSB
  ledcSetup(1, FREQ, RES); // LSB
  ledcAttachPin(MSB_PIN, 0);
  ledcAttachPin(LSB_PIN, 1);
}

// Escreve valor 16-bit
void setVoltage16(uint16_t val){
  uint8_t msb = val >> 8;
  uint8_t lsb = val & 0xFF;
  ledcWrite(0, msb);
  ledcWrite(1, lsb);
}

void loop() {
  setVoltage16(32768); // ~1.65V
  delay(1000);
}
```

Como funciona

val >> 8

Extrai 8 bits superiores
(contribuição principal)

val & 0xFF

Extrai 8 bits inferiores
(ajuste fino: /256)

Circuito necessário:

MSB: R1 = 1kΩ

LSB: R2 = 256kΩ

C = 1μF (filtro)

09

Aplicações Reais

Áudio · Motores · Formas de Onda · Controle de Potência

Aplicações Práticas



Geração de Áudio

DAC ou PWM filtrado

Tons simples, beeps, melodias. DAC para áudio direto, PWM + filtro para qualidade aceitável.

💡 DAC: 8-bit @ 44kHz max



Controle de Motores

PWM direto (sem filtro)

Controle de velocidade DC, servos, drivers de potência. Duty cycle controla velocidade média.

💡 Não precisa de filtro RC!



Geração de Formas de Onda

DAC + DMA ou PWM dual

Senoide, triangular, dente-de-serra. DAC+DMA para waveforms complexas, PWM dual para precisão.

💡 Até ~100kHz com DAC+DMA



Controle Analógico de Potência

PWM 2 canais + filtro

Referência de tensão precisa, calibração de sensores, controle de LEDs com brilho suave.

💡 16-bit com técnica dual

Problemas e Limitações

⚠ Ruído

- DAC 8-bit: SNR $\approx$ 50dB (insuficiente para áudio Hi-Fi)
- Acoplamento de ruído digital via alimentação e GND

⚠ Ripple (Ondulação)

- PWM filtrado sempre tem ripple residual
- Solução: filtro de 2ª ordem (2 estágios RC) ou $\uparrow$ f_{PWM}

⚠ Frequência Inadequada

- f_{PWM} baixa = ripple grande; f_{PWM} alta = menos bits
- Interfere com circuitos sensíveis (EMI)

⚠ Tempo de Resposta do Filtro

- 5τ para estabilizar — lento para sinais rápidos
- Para alta velocidade: usar DAC externo (MCP4725, AD5624)

Resumo — O que Aprendemos

01

DAC

Saída analógica real, 8-bit, simples mas limitado

02

PWM 1ch

Duty cycle + filtro RC, flexível, ripple presente

03

PWM 2ch

MSB+LSB, 16-bit efetivo, alta precisão

04

Filtro RC

Transforma PWM em DC, tradeoff velocidade/suavidade



Guia de Decisão Rápida

Simplicidade → DAC (dacWrite)

Economia → PWM 1 canal + filtro RC

Alta Precisão → PWM 2 canais + filtro

Velocidade → DAC externo (I2C/SPI)

Geração de Sinais Analógicos na ESP32

DAC · PWM · Filtros · Alta Resolução | PlatformIO